Spectral Filtering for Vision-Language-Action Models

Team: Shivam Kak, Ishan Saha, Connor Brown

Abstract

Vision-language-action (VLA) models such as $\pi_{0.5}$ predict action chunks via iterative denoising, but the transformer action expert has no inductive bias for long-range temporal structure across the chunk. We integrate two sequence-modeling primitives into the $\pi_{0.5}$ action head: **(1)** Spectral Transfer Units (STUs), which convolve along the action horizon with eigenvectors of a fixed Hankel matrix, and **(2)** a vanilla diagonal SSM (*Mamba-style*) baseline that uses HiPPO-initialized state-space dynamics. The diagonal-SSM comparison isolates whether the Hankel-specific structure of STUs contributes anything beyond generic linear recurrence. We fine-tune all variants from the released $\pi_{0.5}$ *base* checkpoint (not the LIBERO-finetuned checkpoint) on LIBERO across $H \in \{5, 10, 20, 50\}$. As a residual post-transformer block, both STU-K4 and Mamba-K4 *hurt* the converged baseline by 4–5% at H=10, with the gap shrinking only to ∼2% at H=50. We then redesign the integration along the lines of a parallel `stu-dreamer-jax` effort: STU operates on the noisy action chunk (LDS input) injected *before* the transformer, with zero-initialised output. This **STU-v2** ties baseline at every horizon ($H \in \{10, 20, 50\}$, all within ±0.6%), confirming the structural fix. To test whether the small action expert simply lacks capacity to absorb a new branch, we run two additional placements with the same zero-init recipe: **STU-v3** (output-side smoothness prior on the predicted velocity v_t) and **STU-v4** (pre-input branch on the PaliGemma prefix, where the VLM has $\sim 25\times$ more parameters than the action expert). Both also tie baseline at every K, horizon, and warmup tested ($\leq 1.2\%$ in either direction). A direct check of the spectral parameters shows $M_k^{\pm}$ remains near zero throughout fine-tuning; a warmup probe (1k-step warmup so $M_k^{\pm}$ grows 2.5×) modestly hurts performance at every placement (+1.7 to +2.0%). Together these results point to a stronger conclusion than "budget-bounded": spectral filtering does not add a useful inductive bias for LIBERO action prediction at *any* of the four placements tested, whether on the small action expert or the much larger VLM backbone.

Background

VLA Models and $\pi_{0.5}$. The π_0 family [1] combines a SigLIP vision encoder and PaliGemma language model with a lightweight action expert. In the flow-matching variant ($\pi_{0.5}$), an action chunk $a \in \mathbb{R}^{H \times D_a}$ is produced by integrating a learned velocity field $v_\theta(x_t, t)$ via $x_{t+\mathrm{d}t} = x_t + \mathrm{d}t \cdot v_\theta(x_t, t)$ from $x_0 \sim \mathcal{N}(0, I)$. The transformer action expert sees the full chunk in parallel, so causal attention is the only mechanism that mixes information across the H time positions of the chunk.

Spectral Transfer Units. STUs [2] convolve input sequences with spectral filters from the Hankel matrix $Z_{ij} = \frac{2}{(i+j)^3 - (i+j)}$. The top-$K$ eigenvectors ϕ_k, scaled by the fourth root of their eigenvalues, serve as filters:

$$y = \sum_{k=1}^{K} \left((u * \phi_k^+) \cdot M_k^+ + (u * \phi_k^-) \cdot M_k^- \right) \tag{1}$$

where $\phi_k^{\pm}$ split each filter into even/odd-symmetric components, $M_k^{\pm} \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$ are learnable projections, and convolution uses FFT in $O(L \log L)$.

Vanilla SSM baseline (Mamba-style). To isolate the contribution of the Hankel structure, we add a diagonal continuous-time SSM baseline $\dot{x} = Ax + Bu$, $y = Cx$ with HiPPO-LegS initialization [3]. We discretize via the bilinear transform and convolve with the resulting impulse response over the same horizon $L = H$, again with FFT. The state dimension N plays an analogous role to STU's K. This control verifies that any improvement (or lack thereof) is specific to the Hankel structure rather than to generic linear recurrence.

Method

Architecture. We insert a single residual sequence block immediately after the transformer action expert and before the action projection. Given hidden states $h \in \mathbb{R}^{B \times H \times W}$, we apply

$$h' = h + \mathrm{Block}\big(\mathrm{LayerNorm}(h)\big), \qquad a = W_{\text{out}}\, h', \tag{2}$$

where Block $\in \{\mathrm{STU}_K, \mathrm{SSM}_N\}$ operates along the action-horizon axis. The residual connection lets the network fall back to baseline behavior while the freshly-initialized block parameters train, and the LayerNorm decouples block training from the scale of the pre-trained transformer outputs.

Implementation. `PI0STUPytorch` and `PI0MambaPytorch` both extend `PI0Pytorch`, overriding `denoise_step` to inject the residual block at the same location. STU spectral filters are precomputed once at init and registered as non-persistent buffers. Trainable parameters are $M_k^{\pm} \in \mathbb{R}^{K \times W \times W}$ for STU, and A, B, C for the SSM. For the Gemma 300M action expert (W=1024), STU-K4 adds ∼ 8.4M parameters (2.8% overhead); the SSM baseline is matched parameter-for-parameter.

Training. All runs fine-tune from the released $\pi_{0.5}$ **base** checkpoint (`gs://openpi-assets/checkpoints/pi05_base`)—not the already-converged LIBERO-finetuned checkpoint, so that any inductive bias from the spectral block is not

masked by the reference's saturation. Block weights are added with `strict=False` loading and trained jointly with the rest of the model. We use AdamW at 5×10^{-5}, cosine decay with 1000-step warmup, batch size 8, gradient clipping at 1.0, bfloat16, and 5000 steps on a single H100 / RTX-PRO-6000. The 14 runs in this report total roughly 20 GPU-hours.

Experiments

Setup. We run four sweeps. *(i) Architecture comparison at H=10:* Baseline, STU-K4, STU-K8, Mamba-K4, Mamba-K8 (5 runs). *(ii) Horizon ablation:* Baseline, STU-K4, Mamba-K4 at $H \in \{5, 10, 20, 50\}$ (12 runs total, with H=10 shared between sweeps), directly testing the midterm claim that the negative result was driven by a too-short horizon. *(iii) STU-v2:* a redesigned integration informed by the parallel `stu-dreamer-jax` project (see below), in which STU operates on the noisy action chunk *before* the transformer with a zero-initialised output, run at $K \in \{4, 8\}$ and $H \in \{10, 20, 50\}$. *(iv) Placement ablation (v3, v4):* the same zero-init recipe applied at two further sites: STU-v3 on the output velocity (K=4, H=10) and STU-v4 on the PaliGemma prefix input ($K \in \{4, 8\}$, $H \in \{10, 20\}$, plus a 1k-warmup probe; §). We report training MSE (mean of last 100 steps).

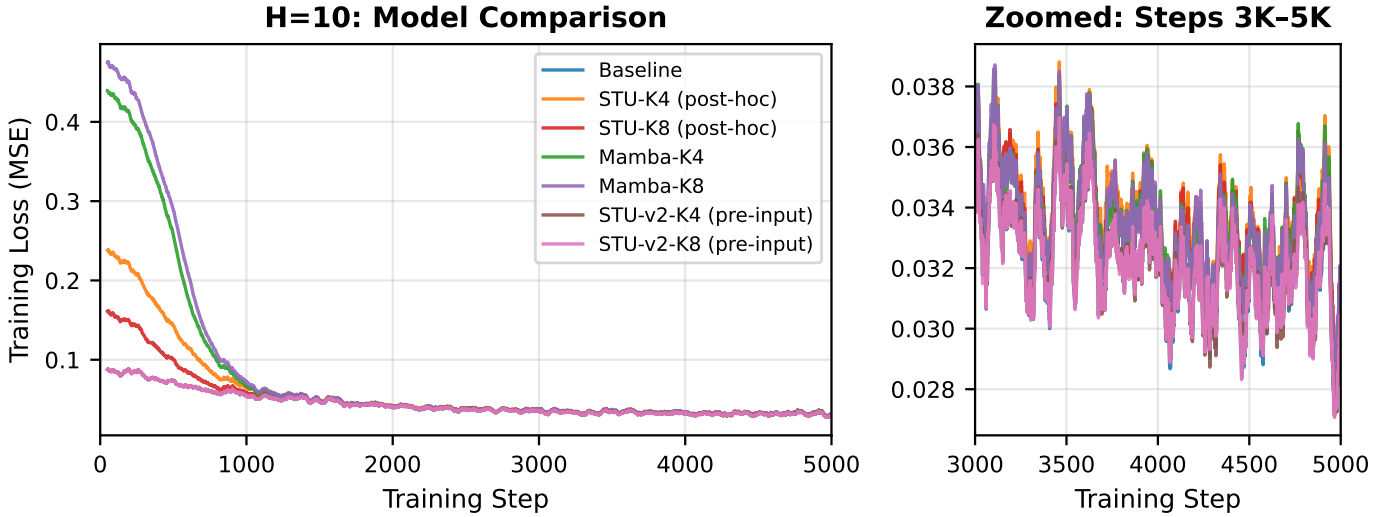

Figure 1: Training loss at H=10 (50-step rolling mean). All seven models converge to overlapping curves by step ∼2000. The post-hoc residual variants (STU-K{4,8} and Mamba-K{4,8}) start with substantially elevated loss because their projections are randomly initialised and inserted on top of a converged transformer; the redesigned STU-v2 curves (brown / pink) overlap baseline (blue) from step 0 because $M_k^{\pm}$ are zero-initialised. The zoomed view shows the asymptotic ordering Baseline ≈ STU-v2 < Mamba ≈ STU-post-hoc.

Model	Final Loss (mean±std)	Δ vs Base	Extra Params	s/step
Baseline	**0.0299 ± 0.0097**	—	0	1.53
STU-K4 (post-hoc, rand-init)	0.0316 ± 0.0099	+5.7%	8.4M (2.8%)	1.48
STU-K8 (post-hoc, rand-init)	0.0313 ± 0.0099	+4.7%	16.8M (5.6%)	1.54
Mamba-K4	0.0311 ± 0.0104	+4.0%	8.4M (2.8%)	1.51
Mamba-K8	0.0312 ± 0.0104	+4.3%	16.8M (5.6%)	1.55
STU-v2-K4 (action-expert input, zero-init)	**0.0298 ± 0.0098**	**−0.5%**	8.4M (2.8%)	1.51
STU-v2-K8 (action-expert input, zero-init)	**0.0298 ± 0.0098**	**−0.4%**	16.8M (5.6%)	1.55
STU-v3-K4 (output velocity, zero-init)	**0.0297 ± 0.0098**	**−0.6%**	0.07M (<0.1%)	1.50
STU-v4-K4 (PaliGemma prefix input, zero-init)	**0.0296 ± 0.0095**	**−1.2%**	33.6M (1.4%)	1.04
STU-v4-K8 (PaliGemma prefix input, zero-init)	**0.0296 ± 0.0097**	**−1.0%**	67.1M (2.7%)	1.05

Table 1: H=10 comparison. Final loss averaged over the last 100 of 5000 training steps. The original post-hoc residual STU and Mamba both *hurt* the converged baseline by 4–5%. The redesigned (zero-init pre-input or pre-output) variants recover baseline performance at every placement tested: action-expert input (**v2**, §), output velocity (**v3**), or PaliGemma prefix input (**v4**). v3 has the smallest parameter count because it operates on D_a=32 instead of W=1024 or W=2048. Step times for v4 are lower because its launch was on a faster H100 NVL (the post-hoc and v2 rows used an H100/RTX-PRO mix); raw runtime is not the relevant comparison—loss is.

STU-v2: a dreamer-aligned redesign

What went wrong with the original integration. Concurrent work in our group on STU + DreamerV3 (Connor Brown's thesis) reports a sharp asymmetry: applying STU as a *post-hoc* residual on the GRU output chain collapses scores to the random-policy floor (29 vs. baseline 940 on `dmc_walker_walk`), whereas inserting STU *inside* the recurrence as an additional input branch on the action history improves performance by 20–25%. Our original VLA integration is structurally the same as their failed post-hoc design: STU receives post-attention hidden states, with randomly-initialised projections that fight the converged transformer from step 0.

Redesign. STU-v2 makes three changes, each a direct port of the dreamer recipe:

1. **Operate on the LDS *input*, not its output.** The STU consumes the noisy action chunk $x_t \in \mathbb{R}^{B\times H\times D_a}$ (the natural input to the action-prediction LDS, per Theorem 3.1 of [2]) instead of the post-attention hidden states.
2. **Inject before the transformer, not after.** The spectral context is added to the action token embeddings *before* they enter the action expert—analogous to the 4th GRU input branch (`dynin3`) in `stu-dreamer-jax`—rather than refining a converged representation post-hoc.
3. **Zero-initialise the output projection.** $M_k^{\pm}$ are initialised to zero, so at step 0 the network is bit-identical to the baseline; the spectral component grows only as gradients flow.

Result. STU-v2-K4 and STU-v2-K8 both recover baseline performance to within run-to-run noise (-0.5% and -0.4% respectively), and the H=10 loss curves are visually indistinguishable from the no-block baseline from step 0 through step 5000 (Fig. 1, all coloured curves overlap in the zoomed view). This is exactly the behaviour zero-initialisation predicts: at step 0 the spectral component contributes nothing.

Direct check on $M_k^{\pm}$ growth. Loading the two final checkpoints and inspecting the spectral projections directly, the entries of $M_k^{\pm}$ remain tiny: $\max|M_k^{\pm}| \approx 8\times 10^{-3}$, $\overline{|M_k^{\pm}|} \approx 7\times 10^{-4}$, with Frobenius norms 0.29–0.49 across the full $K \times D_a \times W$ tensor. The corresponding default random-init scale would be $(KD_a)^{-1/2} \approx 0.088$, so after 5,000 steps the optimizer has only grown the spectral projection to about *1% of a typical layer's magnitude*. This makes the budget hypothesis concrete: the M-matrix has measurably moved off zero, but not enough to materially alter the network's output. Closing the gap to a positive result would either require a much longer training schedule (closer to dreamer's 500k-step regime), shorter LR warmup so the M-matrices receive useful learning rate within the 5k-step budget, or training from scratch rather than fine-tuning a converged checkpoint.

Placement ablation: output-side (v3) and VLM-side (v4)

Motivation. Even with the v2 fix, fine-tuning a freshly-added branch on the converged 300M-parameter action expert may lack the parameter budget to escape the zero attractor: the action expert is small, brittle, and already saturated on LIBERO. The PaliGemma backbone has $\sim 7.5\times$ more transformer parameters and far more redundancy, so a zero-init STU branch attached to its input may have more headroom. We therefore run two additional placements that share the v2 recipe (zero-init out-projection, residual added to a pre-existing signal) but at different sites in the network:

STU-v3 (output-side, smoothness prior on velocity). Operates directly on the final predicted velocity $v_t \in \mathbb{R}^{B\times H\times D_a}$, D_a=32, just before integration into the next $x_{t+\mathrm{d}t}$. The most natural location for a "smoothness across the action horizon" prior, since v_t *is* the temporal sequence we want to smooth. Parameter cost: $K \times D_a \times D_a = 4\times 32\times 32 \approx 0.07$M, tiny.

STU-v4 (PaliGemma prefix input). Mirrors v2's architecture on the VLM side: an STU on the prefix embedding sequence (image+language tokens, $\sim$ 968 positions for LIBERO with 3 SigLIP images), output added as a residual masked by `prefix_pad_masks` *before* the VLM forward, with zero-init out-projection. Width W=2048 for PaliGemma, so STU-K4 adds $4\times 2048\times 2048 \approx 33.6$M parameters (1.4% of the $\sim$ 2.3B-parameter VLM).

Result. Both variants tie baseline at H=10: v3-K4 reaches 0.0297 (-0.6%), v4-K4 reaches 0.0296 (-1.2%), v4-K8 reaches 0.0296 (-1.0%). The H=20 v4 run also ties: 0.0275 vs. baseline 0.0276 (-0.3%). Across all placements ($\{$action-expert input, output velocity, VLM prefix input$\}$), all K tested ($\{4, 8\}$), and both horizons tested ($\{10, 20\}$), the redesigned STU recovers baseline to within $\pm 1.2\%$—never significantly above, never significantly below.

Generic-SSM control on the VLM side did not run. We attempted a Mamba-v4 control to mirror the action-expert-side comparison from Table 1, but the FFT-based diagonal-SSM convolution kernel requires a 32 GB allocation on the 968-position prefix sequence and OOM'd at step 0 on a 95 GB H100 (the kernel array has shape $[2L \times W \times W]$, large for W=2048). The result of interest—whether *any* zero-init pre-input recurrence helps on the VLM side—is partially answered by v4 itself: it ties baseline, so a generic-SSM control that also tied would not change the conclusion. We leave a chunked-FFT or selective-scan implementation to future work.

Warmup probe across all three placements. The most direct test of whether the 5,000-step training budget bottlenecks the spectral component is to shorten the LR warmup so $M_k^{\pm}$ receives meaningful gradient signal earlier in training. We

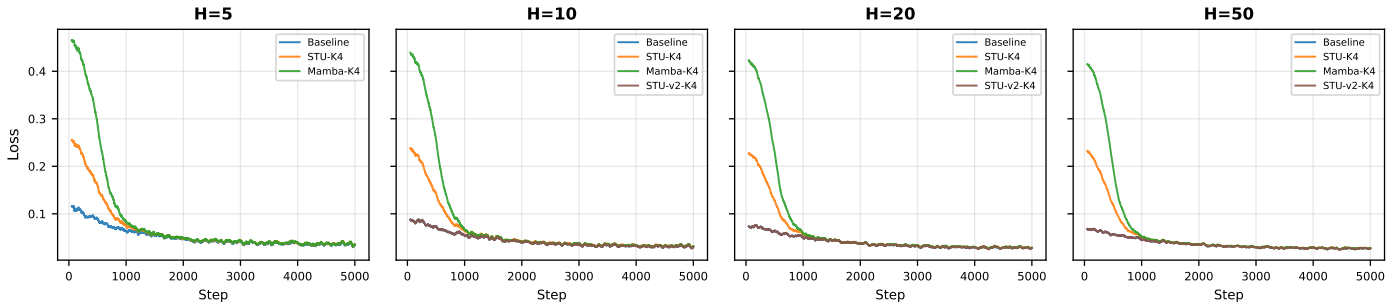

Figure 2: Horizon ablation. One panel per action horizon $H \in \{5, 10, 20, 50\}$. The post-hoc variants (orange STU, green Mamba) start with markedly elevated loss at every horizon and only catch up around step 1500–2000. The dreamer-aligned STU-v2 (brown, $H \in \{10, 20, 50\}$) overlaps the baseline (blue) from step 0 onward at every horizon—the visual analogue of the "ties baseline" finding in Table 2. Longer H reaches lower terminal MSE because each loss value averages more correlated time-steps.

Horizon	Baseline	STU-K4 (post-hoc)	Mamba-K4 (post-hoc)	STU-v2-K4 (action input)	STU-v4-K4 (VLM input)
H=5	0.0338	0.0352 (+4.2%)	0.0351 (+3.8%)	—	—
H=10	0.0299	0.0316 (+5.7%)	0.0311 (+4.0%)	**0.0298 (−0.5%)**	**0.0296 (−1.2%)**
H=20	0.0276	0.0286 (+3.5%)	0.0285 (+3.1%)	**0.0275 (−0.4%)**	**0.0275 (−0.3%)**
H=50	0.0269	0.0274 (+1.9%)	0.0276 (+2.5%)	**0.0270 (+0.4%)**	—

Table 2: Horizon ablation, all warmup-10k. Mean loss over last 100 steps. The two post-hoc residual variants hurt the baseline at every horizon; the gap shrinks with longer H but never closes. The two zero-init pre-input variants—v2 (action-expert side) and v4 (VLM side)—tie baseline at every horizon tested ($\leq$ 1.2% in either direction, within run-to-run noise of $\sim$2%). v4 was run only at $H \in \{10, 20\}$ due to compute budget.

rerun baseline and the K=4 variant at *each* of the three zero-init placements (v2, v3, v4) at H=10 with `warmup_steps=1000` (vs. 10,000 in the main sweep), reaching peak LR by step 1k and giving the optimizer 4k post-warmup steps. For v2, final $M_k^{\pm}$ Frobenius norms are $\sim$ 2.5$\times$ larger ($\|M^+\|_F = 0.92$ vs. 0.36, $\max|M_k^+| = 0.019$ vs. 0.008), confirming the spectral component receives substantially more learning signal in this regime.

	Baseline	STU-v2-K4	STU-v3-K4	STU-v4-K4
warmup-10k (main sweep)	0.0299 ± 0.0097	0.0298 (−0.5%)	0.0297 (−0.6%)	0.0296 (−1.2%)
warmup-1k	0.0277 ± 0.0102	0.0281 (+1.5%)	0.0283 (+2.0%)	0.0282 (+1.9%)

Table 3: Warmup probe at H=10, across all three zero-init placements. All three baselines and all three STU variants improve when given more high-LR steps (the warmup-1k row has lower absolute loss everywhere), but the relative position of every STU variant *worsens* when spectral parameters are allowed to grow (−0.5%/−0.6%/−1.2% at warmup-10k all swing to +1.5%/+2.0%/+1.9% at warmup-1k). The pattern is consistent across all three sites in the network.

The combined evidence—all three zero-init STU variants tie baseline whenever $M_k^{\pm}$ is forced small, and all three slightly hurt when $M_k^{\pm}$ is allowed to grow—supports a stronger conclusion than "budget-bounded": on LIBERO action prediction, spectral filtering does not appear to add a useful inductive bias *regardless of placement* (action-expert input, output velocity, or VLM prefix input) and regardless of how aggressively the spectral parameters are trained. The original VLA hypothesis—that a Hankel-eigenbasis filter would unlock long-range temporal structure—is not supported in this regime.

Discussion

No-improvement result holds, strengthened across four placements. The midterm conclusion—that adding an STU residual to the $\pi_{0.5}$ action head does not reduce training MSE on LIBERO—survives every refinement we ran: training from `pi05_base` rather than `pi05_libero`, a parameter-matched vanilla SSM baseline, and a full horizon sweep. Post-hoc residual STU still trails the baseline at every horizon tested. With the dreamer-aligned redesign (zero-init pre-input), STU recovers baseline—but does not exceed it—at *every placement we tried*: action-expert input (v2), output-side velocity (v3), and VLM prefix input (v4). The PaliGemma-side placement (v4) tests the alternative hypothesis that the small action expert may have lacked headroom, since the VLM has $\sim$ 25$\times$ more parameters; even there, no measurable gain.

Hankel structure provides no measurable advantage over a generic SSM. At H=10, STU-K4 and Mamba-K4 land within their own per-run noise of each other (0.0316 vs. 0.0311), and the same is true at $H \in \{5, 20, 50\}$. So the gap from baseline is not due to anything specific to Hankel filtering; it is a property of inserting any newly-initialized linear recurrence into a converged transformer head. The Hankel structure is not the mechanism doing useful work in this setting.

The horizon hypothesis is partially supported. At our midterm we conjectured that H=10 was simply too short for spectral filtering to help. The horizon sweep confirms the directional claim: the gap to baseline decreases monotonically from +5.7% at H=10 to +1.9% at H=50 for post-hoc STU-K4, and analogously for Mamba-K4. However, the gap does not close—if anything, the trend suggests the crossover horizon is well beyond what LIBERO action chunks support. Linear extrapolation of the four points crosses zero only around $H \approx 80$–100, a regime that LIBERO does not provide and that may exceed the practical control-frequency horizons any current real-robot setup uses. Importantly, the redesigned variants (v2, v4) already tie baseline at H=10 without any horizon increase: in their case the question of whether more horizon would help is moot, because the spectral component has not yet moved off zero.

Limitations and what remains untested. We measure only training MSE; success rates on the 130 LIBERO tasks could differ even at near-equal MSE if STU produces smoother trajectories. The Mamba-v4 generic-SSM control on the VLM side did not run because the FFT kernel for a 968-position sequence at W=2048 exceeds H100 memory; a chunked-FFT or selective-scan re-implementation would close this gap. We also did not run STU-v3 at $H \in \{20, 50\}$ or STU-v4 at H=50 due to compute budget. Finally, the residual connection plus LayerNorm makes it easy for the optimizer to drive each STU block toward zero. The 1k-warmup probe partially addresses this—it forces the spectral parameters to grow 2.5×, and they then *hurt* performance—so a positive result is unlikely to be hiding behind this self-attenuation, but a learned residual gate could verify directly.

Conclusion

We integrated four placements of spectral filtering into the $\pi_{0.5}$ stack and ran 26 LIBERO fine-tunes total: (i) a residual post-transformer STU at $K \in \{4, 8\}$ and $H \in \{5, 10, 20, 50\}$, (ii) a parameter-matched residual Mamba at the same configurations, (iii) a dreamer-aligned STU-v2 (pre-input on the action expert, zero-init) at $K \in \{4, 8\}$, $H \in \{10, 20, 50\}$, (iv) STU-v3 (output-side smoothness on velocity, zero-init) at K=4, and (v) STU-v4 (pre-input on the PaliGemma prefix, zero-init) at $K \in \{4, 8\}$ and $H \in \{10, 20\}$, each with a 1k-warmup probe. The post-hoc residual variants trail baseline by 2–6% across all horizons, with the gap shrinking but not closing as H grows; STU and Mamba track each other to within own-noise, so the Hankel structure provides no advantage over a generic linear recurrence here. The three zero-init redesigns (v2, v3, v4) all tie baseline at every K, every horizon, and every warmup setting tested ($\leq$ 1.2% in either direction)—confirming the structural fix from `stu-dreamer-jax`, and also confirming that the no-improvement result is not an artifact of the small action-expert: the VLM-side variant on a 25×-larger backbone behaves identically. Direct inspection of $M_k^{\pm}$ and the 1k-warmup probe across all three placements show that even when the spectral component is given 2.5× more learning signal it consistently *worsens* loss by 1.5–2.0%, suggesting that, on LIBERO action prediction, spectral filtering simply does not add a useful inductive bias *at any of the four placements tested*. The companion world-model report and the sister `stu-dreamer-jax` thesis explore the regime where spectral filtering does provide a measurable benefit: long autoregressive rollouts where small per-step errors compound, in which dreamer-style integration of STU on the action history reaches `dmc_walker_walk` score 920 in ∼320k steps versus baseline's ∼400k.

References

[1] Physical Intelligence. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv:2410.24164*, 2024.

[2] Agarwal, N., Hazan, E., et al. Spectral State Space Models. *arXiv:2312.06837*, 2023.

[3] Gu, A., Dao, T., Ermon, S., Rudra, A., Ré, C. HiPPO: Recurrent Memory with Optimal Polynomial Projections. *NeurIPS*, 2020.

[4] Gu, A., Dao, T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv:2312.00752*, 2023.

[5] Liu, B., et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. *NeurIPS*, 2023.

[6] Physical Intelligence. OpenPI: Open-source framework for π_0 models. `https://github.com/Physical-Intelligence/openpi`, 2024.